דוח משוב לסטודנט

מזהה סטודנט	[redacted]
מספר מבחן	0001
תאריך בחינה	יום שלישי, 1 ביולי 2025
מזהה קורס	20594
שם קורס	מערכות הפעלה
מרצה	אריה לויטן

תשובות נכונות (שאלות סגורות)	ניקוד שאלות סגורות	ניקוד שאלות פתוחות	ציון מבחן מקורי	**ציון מבחן סופי**
6	30.00	60.00	90.00	**90.00**

סיכום

מספר שאלה בטופס זה	מספר השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור	ניקוד	ניקוד מירבי	התשובה שסומנה	התשובה הנכונה
1	1	1	5.00	5.00	✓ 1	1
2	2	3	5.00	5.00	✓ 3	3
3	3	3	5.00	5.00	✓ 3	3
4	4	2	5.00	5.00	✓ 2	2
5	5	2	5.00	5.00	✓ 2	2
6	6	1	5.00	5.00	✓ 1	1
7	7		15.00	25.00		
8	8		15.00	15.00		
9	9		15.00	15.00		
10	10		5.00	5.00		
11	11		5.00	5.00		
12	12		5.00	5.00		

✓ תשובה נכונה ✓ תשובה חלקית ✗ תשובה שגויה ⊘ לא נענתה ⓘ תשובה עודפת

האוניברסיטה
הפתוחה

דף תשובות

מספר גרסה	מספר תעודת זהות	מספר הקורס	תאריך בחינה
0001	[redacted]	20594	010725

0	●	●	●	○
1	○	○	○	●
2	○	○	○	○
3	○	○	○	○
4	○	○	○	○
5	○	○	○	○
6	○	○	○	○
7	○	○	○	○
8	○	○	○	○
9	○	○	○	○

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימון, יש לסמן X על התשובה הלא נכונה - ⊗
   ולהשחיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה,
   ולהשחיר מחדש את סימן ה- ⊗ באופן שיכסה המחיקה - ●

שאלה / תשובה	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	●	○	○	○	○	●
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	●	●	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	●	●	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	40	39	38	37	36	35	34	33	32	31	30	29	28	27	26	25	24	23	22	21
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	60	59	58	57	56	55	54	53	52	51	50	49	48	47	46	45	44	43	42	41
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

[redacted]

[redacted]

יש לרשום את כל תשע הספרות

האוניברסיטה הפתוחה

ה' בתמוז תשפ"ה

מס' שאלון - 519

1

ביולי 2025

סמסטר 2025ב

20594 / 4

מס' מועד 83

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם
בתשובה יחידה הנכונה ביותר מבין התשובות.
את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.

ג. בחלקים ב' ו ג' מופיעות שאלות פתוחות.
ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ.
אין חובה להשתמש בכל השורות המוקצות לצורך התשובות,
אך אין לחרוג מהמקום המוקצה.

חומר עזר:

כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

הנחיות למבחן

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א׳ יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב׳ ו ג׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות, בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

איזו משיטות תזמון SCHEDULING הבאות יכולה לגרום להרעבה?

[✓ א.] shortest job first ← יכולות להגיע הרבה עבודות קצרות כך שתהיה הרעבה לעבודה ארוכה שמחכה

ב. first come first served

ג. Round robin עם quantum קצר

ד. Round robin עם quantum ארוך

שאלה 2

← הרצת VM מאפשרת דימוי מחשב מרוחק עם מ"ה משלו, הוא משתמש בהרבה משאבים

תארו את השתלשלות הפעולות בביצוע SYSTEM CALL כתיבה לקובץ ע״י תהליך המשתמש במערכת LINUX ווירטואלית אורחת שרצה על תוכנת VMWARE המשתמשת שיטת וירטואליזציה VT ומופעלת על מערכת WINDOWS. ציינו איזה KERNEL ״מנסה״ לבצע את הפקודה ומה קורה בפועל.

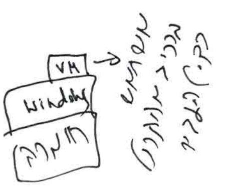
Host
hypervisor סוג 2
זה לא רלוונטי
כאשר משתמשים ב-VT
[illegible]

~~א.~~ ה-SYSTEM CALL מתבצע ע״י KERNEL של מערכת אורחת GUEST עד הסוף(כולל כתיבה בפועל) אם למערכת אורחת יש מקום בדיסק משלה.

ב. ה-SYSTEM CALL מתבצע ישירות ע״י KERNEL של מערכת מארחת HOST בגלל שבזמן התקנת מערכת אורחת (LINUX), VMWARE החליפה את כל קריאות המערכת שלה לקריאות מקבילות של WINDOWS.

[✓ ג.] ניסיון כתיבה לדיסק יגרום להפעלת מודול MODULE ב-KERNEL של מערכת המארחת HOST שנוסף שם ע״י VMWARE בזמן ההתקנה או היה שם קודם, והוא מעביר אותה ל KERNEL המארח HOST.

ד. ניסיון כתיבה לדיסק יגרום להפעלת מודול MODULE ב-KERNEL של מערכת האורחת GUEST שנוסף שם ע״י VMWARE בזמן ההתקנה או היה שם קודם, והוא מבצע את הכתיבה בפועל.



קיפאון – מניעה הדדית (משאב פרטי)
– אין חטיפה רשות
– המתנה ובקשה – מחזיק משהו ומבקש אחר נוסף
– מעגל

שאלה 3

במערכת קיימים כמה סוגי משאבים עם מספר עותקים משתנה של כל משאב.

באיזה מהמצבים הבאים בהם קיים מעגל בגרף הקצאות **המשאבים בפועל**, ניתן לומר **בוודאות** שיש קיפאון deadlock ?

א. כשלפחות עותק **מאחד המשאבים המוקצים** לכל אחד מתהליכים במערכת הוא חלק מהמעגל.

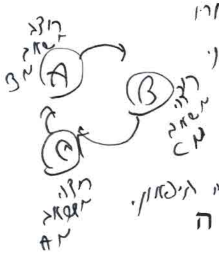
ב. כשלפחות עותק אחד של **כל משאב** במערכת הוא חלק מהמעגל.

[✓ ג.] כשלכל אחד מהמשאבים במערכת יש רק מופע אחד.

ד. כשלכל אחד מהמשאבים במערכת יש יותר ממופע אחד.

deadlock עם מעגל זה כאשר כל תהליך
מחכה למשאב מהתהליך האחר
כל אחד מחכה למשאב מהשני
ומכיוון שיש רק 1 מכל סוג
זה (משאב הוא בלעדי
אז אין לתהליך המחכה מקור
אחר לקבל אותו וכך יהיה הקיפאון.
2^{32}

אף הוא עצמו
מחזיק משאב
שתהליך אחר רוצה



שאלה 4

נתונה טבלת הדפים של תהליך במערכת, אורך הכתובת הוירטואלית 32 סיביות ואורך שדה ה OFFSET הוא 11 סיביות. כל המספרים בטבלה דצימליים וכל הכתובות של בייית בזיכרון.

$2^{11} = 2048$ גודל דף

Frame Number	Valid bit	Page Number
1	1	0
2	1	1
0	1	2
0	1	3
3	0	4
0	0	5

לאיזו כתובת פיזית, אם ניתן לחשב, תמופה הכתובת הוירטואלית הבאה : 5000 ?

דף 0 : ~~[illegible]~~ 2048–0
דף 1 : 4096–2048
דף 2 : 6144–4096

$\frac{5000}{2048} = \lfloor 2.44 \rfloor = 2$

א. 5000

[✓ ב.] 904

ג. 7048

ד. לא ניתן לחשב

שאלה 5

מגדיל את מקביליות המערכת, מפחית את זמן המעבד הנדרש לפעולה

מהי הסיבה העיקרית לשימוש ב- DMA?

~~א.~~ למעבד דרוש פחות זמן לביצועה פקודה חישובית אחת.

[✓ ב.] הקטנת הזמן שלוקח למעבד בסה"כ לטפל בפעולות קלט-פלט.

ג. למעבד דרוש פחות זמן לביצועה גישה לזיכרון הראשי.

ד. מאפשר להתקן, למשל דיסק, להשתמש בזיכרון הראשי כמטמון.

נא לא לכתוב בשולים

נא לא לכתוב בשוליים

שאלה 6

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים:

$\frac{2^{11}}{2^{2}} = 2^{9} = 512$
רשומות

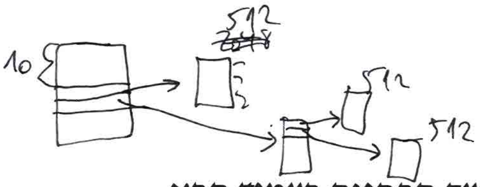
- גודל הבלוק במערכת הקבצים הוא 2 Kbytes $2^{11} = 2048$
- כתובת הבלוק היא 4 בתים (bytes) 2^{2}
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

חשבו כמה בלוקים בסה"כ יתפוס קובץ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו) בגודל 3145700 בית(3 מגה בית פחות 28 בית) .

הערה: לצורך החישוב 1 KByte = 1024 Byte , 1 MByte = 1024 KByte

[✓ א.] 1540 ~~Kbytes~~
ב. 1539 ~~Kbytes~~
ג. 1541~~Kbytes~~
ד. 1536~~Kbytes~~

$\frac{3145700\,b}{2048} = 1535.98$
בלוקים
~ 1536



המשך הבחינה בעמוד הבא

$10 + 1 + \cancel{2048}\, 512 + 1 + 1 + 1$

$1536 + 4 = 1540$

חלק ב (55 נקודות)

ענו על שאלות 7-9.

שאלה 7 (25 נקודות)

א. (10 נקי)

הוצע שינוי לבעיית N פילוסופים סועדים ו N מזלגות: כל פילוסוף יכול להשתמש בכל שתי מזלגות פנויים שהן לא בהכרח לימינו או שמאלו. ממשו(ב C או פסודוקוד) את הפונקציות הבאות בצורה יעילה ומאפשרת לכמה שיותר פילוסופים לאכול במקביל ללא אפשרות קיפאון. כשאין מזלגות פנויים, פונקציה take_forks() ממתינה עד שיתפנו. ניתן להשתמש במבני נתונים, משתנים ואמצעי סנכרון לפי הצורך ולהסביר את הבחירה מבחינת היעילות.

n1 ו n2 הם מספרי מזלגות, i – מספר הפילוסוף.

פילוסוף i לוקח 2 מזלגות

- take_forks(i)
- put_forks(int n1,int n2)

החזרת 2 מזלגות

תשובה והסבר:

```
mutex m1 = 1
int counter = N

take_forks(i):
 while true:
     down(m1)
     if(counter >= 2)
         counter = counter - 2
         up(m1)
         exit;
     up(m1)
     ~~while(counter<2)~~
     ~~{busy wait}~~

put_forks(i)
    down(m1)
    counter = counter+2
    up(m1)
```

חסר מעקב מזלגות פנויים

הסבר: נשתמש במשתנה counter שמאותחל ל-~~N~~ N שיציג את מספר המזלגות. כל פילוסוף צריך 2 מזלגות (N פילוסופים צריכים 2N מזלגות)

ונשתמש במנעול mutex שמאותחל ל-1 (פתוח) ~~א~~ נשתמש ב mutex ולא בסמפור כי הוא יותר יעיל (סמפור דורש מעבר למצב גרעין) המנעול יגן על המזלגות כך שלא ייווצר מצב שיילקחו יותר מזלגות ממה שיש בפועל מבלי Context switch.

ב take_forks אנו מבצעים לולאה אינסופית (while true). ננעל את m1 ע"י down(m1) כך שנמנע גישה למשתנה counter משאר הפילוסופים ונבדוק האם counter>=2 ז"א האם יש לפחות 2 מזלגות פנויים, אם כן "ניקח אותם" ע"י הורדת counter ב2, פתיחת המנעול ויציאה מהפונקציה. אם לא היו לפחות 2 מזלגות פנויים נעלה את המנעול m1 (נפתח אותו לשימוש) ונישאר בתוך לולאת ה while true עד שיתפנו מספיק מזלגות וניקח אותם.

ב Put_forks רוצים להחזיר מזלגות לכן ננעל את הגישה למונה עם m1, נעלה את המונה ב2 (נחזיר את המזלגות) ונפתח את המנעול 5 על המונה.

(המשך השאלה בעמוד הבא)

ב. (15 נקי)

התייחסו לבעיית N פילוסופים סועדים עם N מזלגות עם השינוי הבא:
כדי לאכול כל פילוסוף **יכול** להצטרך ל 2 מזלגות או רק למזלג אחד- **ימין**. פילוסוף שקרא לפונקצית לקיחת מזלג ימין בלבד take_right_fork(int i) , לא יוכל לקרוא לפונקציה ללקיחת 2 מזלגות עד שיניח את המזלג ע״י put_right_fork(int i) . משתנה i הוא מספר פילוסוף.

ממשו (ב C או פסודוקוד) את הפונקציות הבאות בצורה הקצרה והיעילה ביותר שתבטיח לכל הפחות ל N/2(N זוגי) פילוסופים לאכול במקביל ללא אפשרות קיפאון. ניתן להשתמש באמצעי סנכרון לפי הצורך ולהסביר את הבחירה מבחינת היעילות.

- take_right_fork(int i)
- put_right_fork(int i)
- take_forks(int i)
- put_forks(int i)

תשובה והסבר:

```
mutex m_1 = 1
int counter = N
int lock[N]=0
```

מערך באורך מס׳ הפילוסופים מאותחל ל-0

```
take_right_fork(i):
while true:
        down(m_1)
        if (counter > 0)
                counter=counter-1
                up(m_1)
                lock[i]=1
                exit
        up(m_1)
```

```
put_right_fork(i):
    down(m_1)
    counter=counter+1
    up(m_1)
    lock[i]=0
```

חסר מעקב מזלגות פנויים

אפשר לקחת רק ליד פילוסוף, אחרת בנאלי

```
take_forks(i):
~~while(lock[i]~~=
while true:
        ~~while~~ if (lock[i]==0)
            down(m_1)
            if(counter>=2)
                counter=counter-2
                up(m_1)
                exit
        up(m_1)
```

```
put_forks(i)
  down(m_1)
  counter=counter+2
  up(m_1)
```

הסבר בעמוד הבא

נא לא לכתוב בשוליים

נשתמש ב m_1 mutex שיוותחל ל-1 (פתוח) להגנה מונה המזלגות. ומ Counter
שיוותחל לN כמס' המזלגות הפנויים בהתחלה ומערך lock[N] מערך בגודל
N כמות הפילוסופים שכל המנעולים שלו יוותחלו ל-0 (פתוח) ויהוו הגנה רק שפילוסוף לא יוכל
לקחת מזלג ימין אחד ולאחר מכן לקחת זוג מזלגות (ננעל אותו בעת לקיחת
מזלג ימני ונפתח אותו רק לאחר החזרתו).

take_right_fork – ~~[illegible]~~ הפונ' רצה בלולאה אינסופית (while true) עד ללקיחת המזלגות.
ננעל את m_1 ונבדק את הגישה למונה המזלגות הפנויים ונבדוק שקיים לפחות מזלג אחד פנוי.
אם כן, ניקח אותו, נפתח את הנעילה ונסמן את המנעול שלו במערך lock ב-1 (נעול) הנעילה הזו
משמעותה שאינו יכול לקחת זוג מזלגות עד שלא יחזיר את המזלג הימני.

put_right_fork – מחזיר את המזלג הימני ופותח את הנעילה של lock

ב takeforks & put_forks עלים בדיוק זהה רק שב take_forks מוודאים שאכן lock[i] נעול,
רק אם הוא פתוח יאפשרו לקחת זוג מזלגות

15
(7)

אל תכתוב בשוליים

נא לא לכתוב בשוליים

שאלה 8 (15 נקודות)

א. (5 נק')

כחלק מאלגוריתם פינוי דפי זיכרון, מערכת הפעלה נתונה עבור כל מסגרת פיזית מנהלת מונה המייצג את מספר פניות אליה. האם עבור המסגרות המאחסנות דפים משותפים בין תהליכים SHARED MEMORY נדרש מונה יחיד או מספר מונים(לפי תהליך)? הסבירו את היתרונות והחסרונות של כל אפשרות אם יש.

תשובה והסבר:

מונה יחיד קל לתחזוקה ועולה כי הוא יש צורך בדף זה לכן. מבחינת הדף לא משנה אם כמה תהליכים פונים אליו זה אותו הדבר פונים אליו ומשנים תדירות (R=0, R=1, R=2). היתרון בשיטה זו הוא שנחשב הוא לדף יש צורך ולא משנה ע"י כמה תהליכים) ונשאיר אותו. במס' מונים יש חסרון שבו תהליך מסוים יפנה לדף רק פעם 2 לדף יסופר אותו לא והאלגוריתם יזהה צורך ויפנה את הדף אך תהליך נוסף פונה בתדירות גבוהה לאותו הדף וצריך אותו והפנוי יגרום כי (יתבצע pagefault) והוא יצטרך להביא את הדף 6 פעם מחדש. (היתרון במס' מונים זה שאם רק תהליך מסוים פונה לדף לעתים אז ברגע שהתהליך זה יפסיק לרוץ נוכל ישר לפנות את הדף (האפשרות השונה) וזה לא יראה לי תמונה שגויה)

ב. (5 נק')

כחלק מאלגוריתם פינוי דפי זיכרון מערכת הפעלה נתונה מנהלת מונה המייצג את מספר פניות לכל דף (PAGE) זיכרון. המונה נמצא בטבלת הדפים. מה אמור להיות ערך המונה (ביחס לתהליך האב PARENT) עבור דפי זיכרון בעקבות ביצוע ()fork במערכת המנהלת את הזיכרון על בסיס copy_on_write? הסבירו יתרונות חסרונות של כל אפשרות אם יש.

תשובה והסבר

במערכת על בסיס COW לאחר ביצוע fork גם האב וגם הבן חולקים דפי זיכרון משותפים ושניהם יכולים לגשת אליהם. אך כאשר הבן ירצה לבצע שינוי יווצר עותק נוסף של אותו דף שהוא רוצה לשנות ואת העותק הבן ישנה. ערך המונה יהיה מס' הפניות של האב + מס' הפניות של הבן (פניות R ולא M אשר מבוצעות על אותו דף) היתרונות והחסרונות דומים למונה יחיד ומס' מונים כמו בסעיף א' אך כאשר יתבצע COW ע"י הבן לו יהיה דף נפרד ומונה נפרד (הדף עצמו כבר יהיה שונה) ויהווה דף אחר

(המשך השאלה בעמוד הבא)

נא לא לכתוב בשוליים

ג. (5 נק')

בהתייחס לסעיפים א ו-ב, האם קיים הבדל בניהול אזורי זיכרון המאחסנים קוד התוכנית והנתונים?

תשובה והסבר:

בהתייחס לשני תהליכים עם sharedmemory קוד התוכנית גם תהליך יאוחסן בזיכרון של אותו תהליך ויבצע הפניה לכך המשותף במרחב הזיכרון המשותף ולמעשה כל תהליך יהיה טבלת דפים וירטואלית עם דף וירטואלי כלשהו אשר יקושר לאותה מסגרת פיזית של הדף בזיכרון המשותף)

ובמקרה של אבא ובן (בדומה [illegible]), תהליך לאחר ה fork יהיו חלוקים את איזור זיכרון (תהליכונים) והנתונים כוונתי יקום אך ברגע שהבן רוצה לשנות דף כלשהו יבוצע COW וייצר עותק לבן וכך יבוצעו השינויים שלו

15
(8)

שאלה 9 (15 נקודות)

כידוע, אחת הבעיות הפוגעות בביצועי מערכת הקבצים היא קבצים שאינם שמורים באופן רציף על הדיסק הקשיח. **הסעיפים הבאים מתייחסים להקצאת מקום לקבצים בעזרת אינדקס (I-Node).**

א. (5 נק')

האם הקצאת מקום רציף מועילה לגישה רציפה לקובץ, לאקראית(ישירה) או לשניהם?

תשובה והסבר:

מועילה לשניהם. הקצאה רציפה בגישה רציפה תוכל לשמור רצף על הבלוקים (פחות תזוזות דיסק מאשר אם לא היו רציפים)

ובגישה אקראית אמנם עדיין תהיה תזוזה לפי המצביעים אך החוט רצוף ולכן עדיין התזוזות יהיו קרובות יותר אם היה בלוק אחד בהתחלת הקובץ ובלוק אחד בסופו ([illegible] לכל הבלוקים)

(המשך השאלה בעמוד הבא)

נא לא לכתוב בשוליים

נא לא לכתוב בשוליים

ב. (10 נקי)

הוצעו שתי שיטות לייעול מערכת קבצים:

1. השארת מקום ריק (רווח) בגודל קבוע לאחר כל קובץ בזמן יצירתו, כך שאם הקובץ צריך לגדול, הוא יכול להתחיל לגדול באופן רציף. המקום השמור לאחר כל קובץ מיועד לקובץ עצמו, אבל מותר להשתמש במקום זה עבור הקצאה לקבצים אחרים אם אין ברירה אחרת (כלומר רק במקרה שאין יותר מקום פנוי על הדיסק חוץ ממקומות השמורים לגדילה של קבצים).

2. הקובץ הראשון יישמר בתחילת הדיסק. בהמשך כל קובץ חדש יישמר החל מאמצע השטח הרציף הארוך ביותר שנשאר בדיסק.

השוו בין שתי השיטות ופרטו את היתרונות והחסרונות של כל אחת.

15
(9)

תשובה והסבר:

שיטה 1: בשיטה זו היתרון זה שהיא מאפשרת הקצאה רציפה של הקובץ אך החסרונות הן שיכול להיות שהמקום שהוקצה לא יספיק ותהיה הקצאה במקום אחר (מה שיגרור הקצאה לא רציפה לאותו קובץ ואולי גם לאחר זמן ישתמשו במקום שהוקצה לקובץ אחר) וגם שהגודל המוקצה יהיה גדול מידי עבור אותו קובץ וזה יגרור ריסוק פנימי (אלא אם קובץ אחר ישתמש)

שיטה 2: בשיטה זו היתרון הוא שהיא נותנת מקום לקובץ להמשיך הקצאה רציפה שלו אך החסרון הוא שיכול להיות ריסוק חיצוני של אזורים בין הקבצים שיישארו לא מוקצים. ⟵

בהשוואה בין השיטות נאמר בשיטה מתאים לקבצים שאם ידועים עליהם פרטים מראש כמו קבצים עם טווח גודל מסוים שאפשר לתת רווח להקצות להם (שיטה 1) וכך (נמנע את החסרונות.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

חלק ג (15 נקודות)

ענו על שאלות 10-12. (5 נקודות לשאלה; 15 נקודות לחלק זה).

שאלה 10

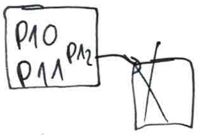
הסבירו איך מעבד יודע איפה נמצא הקוד שעליו לבצע בעקבות ביצוע TRAP עם מספר מסוים, לדוגמא int 80h .

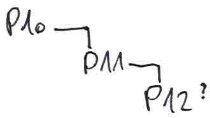
5

(10)

תשובה והסבר:

במצב משתמש מתבצע system call אשר דורש מעבר למצב מיוחס לביצוע הפקודה. לכל קריאת מערכת יש מס׳ (int ולא str) ייחודי לה, אותו מס׳ מזהים את קריאת המערכת הספציפית בווקטור הפסיקות, הוא יודע מה עליו לבצע במצב kernel מכיוון ומס׳ קריאת מערכת שבווקטור הפסיקות. המעבד יודע איפה נמצא הקוד שעליו לבצע כעת, וקטור הפסיקות נגיש כוקטור הבא למס׳ שמזהים את הקריאה ומשם נגישים למיקום הקוד שעליו לבצע ולסיום מוחזר עם סיום למשתמש (מצב משתמש).





שאלה 11

מה יהיה הפלט אחרי הרצת הקוד הבא במערכת XV6 שתומכת בקונטיינרים?
ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת הקוד היה 10 .
ניתן להניח שכל קריאות מערכת מצליחות.

```
10
pid = fork();
if (pid == 0)  {                                              בן
    pid = fork();
    unshare(PID_NS);                                          אין fork אחריו לא משפיע
                                                                                12
    if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n",
pid);
    else  {
                                                                          11
       printf (stdout, " Child's perspective on his parent: PID=%d \n", getppid());
       printf (stdout, " Child's perspective on himself: PID=%d \n", getpid());
                                                                    12
     }
}
                                                                              11
else  (stdout, "Grandparent's perspective on his child: PID=%d \n", pid);     [illegible]
```

5

(11)

תשובה והסבר:

פלט: Parent's...his child: PID=12, Child's...parent: PID=11, Child's...himself: PID=12, Grand...PID=11

זה הפלט תלוי בתזמון. הסבר: ~~[illegible]~~ בתחילת ההרצה, הPID הגדול ביותר היה 10. מבוצע fork אשר יוצר תהליך חדש (הבן הוא 0 והאב רואה אותו כ11) נכנס ב if הראשון pid==0 בתור הבן ושם נבצע שוב fork ויצירת תהליך חדש שהאב שלו יראה את הpid כ12. שורת ה unshare לא תשפיע (לא יווצר לנו מרחב שמות חדש עם מספור מ-1) מכיוון שלא קורא fork אחרי אותה שורה, לכן נמשיך כרגיל. נכנס ל if של pid>0 ~~[illegible]~~ הוא ונדפיס איך שהאב רואה את בנו בתור PID=12 לאחר מכן ניכנס ל else של הבן ונדפיס איך הבן רואה את אביו (PID=11) ואת עצמו (PID=12) לאחרונה נכנס ל else של ה if הראשון ~~[illegible]~~ (האב המקורי (הסבא)) ונדפיס את איך שהוא רואה את בנו (PID=11)

יש לענות על שאלה 12 אחת בלבד בהתאם לשנת הלימודים

שאלה 12 לנבחנים שלמדו ב 2025

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
unshare (PID_NS);
unshare (MOUNT_NS);
int pid = fork();
if (pid == 0) {
   mkdir ("MDir");
   mount ("internal_fs_b","MDir",0); //mount a preformatted internal_fs_b
   int fd = open ("MDir/file.txt",O_CREATE|O_RDWR);
   write (fd ,"Some text\n",10);
   close (fd);
  }
If (pid>0) {
    sleep (10000); // wait for child to finish
    if (open ("MDir/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to
open\n");
    else printf (stderr, "father succeeded\n");
}
if( pid == 0) {
    if (open ("MDir/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to
open\n");
    else printf (stderr, "son succeeded\n");
  }
```

(האב מחכה שהבן יסיים ורק כשיגש לקובץ הוא כבר יהיה קיים.

5
(12)

תשובה והסבר:

יודפס father succeeded Son succeeded. הסבר: תחילה בוצע unshare ל NS של Mount ולכן נוצר מרחב מבודד חדש שבו PID יתחילו מ1. כאשר הראשון שיווצר יבצע mount חדש (עותק לכל הקבצים) שיבצעו בו התהליכים שלא באותו mount לא יוכלו לראות קבצים שנוצרים בו. מכיוון שכל אחד הפקודות מבוצעות אחרי הפקודות אלו אז בעצם 6 מה שיקרה הוא שכולם יבצעו באותו מרחב מבודד ולכן כאשר הבן ייצור קובץ במרחב הזה גם הבן וגם האב יוכלו לגשת אל אותו קובץ ולפתוח אותו (ולכן שניהם ידפיסו הצלחה)

שאלה 12 לנבחנים שלמדו ב 2024

האם בשביל מימוש מנגנון קונטיינרים נדרשת תמיכת חומרה מעבר למה שקיים במערכת סטנדרטית?

תשובה והסבר:

בהצלחה!

נא לא לכתוב בשוליים